

Multi-Tenant Personal Budgeting SaaS (MVP)

Peran: Senior Technical Product Manager & Cloud Architect • Status: Draft MVP

1. Problem Statement & Value Proposition

Problem Statement

Mengelola keuangan pribadi sering kali menjadi hal yang membingungkan bagi banyak individu karena pencatatan yang tersebar dan kurangnya visualisasi arus kas secara *real-time*. Di sisi lain, ketika beralih ke solusi digital, kekhawatiran terbesar pengguna adalah **keamanan dan privasi data finansial** mereka. Banyak platform gagal memberikan jaminan transparansi arsitektur bahwa data satu pengguna tidak akan pernah terlihat atau bocor ke pengguna lain. Bagi penyedia layanan (SaaS provider), membangun infrastruktur database terpisah untuk setiap pengguna (*silos database*) memerlukan biaya operasional yang sangat tinggi dan manajemen infrastruktur yang rumit, terutama pada fase MVP.

Value Proposition

Aplikasi *Personal Budgeting SaaS* ini menawarkan platform pengelolaan keuangan yang ringan, cepat, dan intuitif dengan pendekatan arsitektur **Multi-Tenant Shared Database via Logical Isolation**.

Dengan memanfaatkan teknologi **Supabase Row Level Security (RLS)** pada database PostgreSQL, platform ini menjamin isolasi data yang ketat setingkat bank (*bank-grade security*) di mana setiap *tenant* (pengguna/workspace) hanya dapat mengakses datanya sendiri. Solusi ini memberikan efisiensi biaya infrastruktur yang luar biasa bagi penyedia layanan karena berjalan di atas satu database tunggal, namun tetap memberikan ketenangan pikiran (*peace of mind*) mutlak bagi pengguna terkait privasi data mereka.

2. Minimum Viable Product (MVP) Features

Aplikasi MVP ini berfokus pada efisiensi fungsionalitas dengan membatasi fitur hanya pada 3 fungsi inti:

A. Autentikasi & Registrasi Tenant

Mekanisme gerbang utama untuk mengidentifikasi dan memisahkan *workspace* antar pengguna secara unik.

- **Registrasi & Login:** Menggunakan Supabase Auth dengan metode Email & Password.
- **Tenant Initialization:** Setiap kali pengguna baru mendaftar, sistem secara otomatis menghasilkan UUID unik (`user_id`) yang bertindak sebagai ID Tenant global mereka.
- **Session Management:** Token JWT (JSON Web Token) disimpan secara aman di sisi *client* (Vite + React) untuk memvalidasi setiap *request* ke Supabase.

B. Manajemen Transaksi

Fungsionalitas utama untuk melacak pergerakan uang masuk dan keluar.

- **Pencatatan Transaksi:** Pengguna dapat melakukan operasi CRUD (Create, Read, Update, Delete) pada data transaksi secara mandiri.
- **Atribut Data:** Setiap transaksi wajib memiliki nominal angka, tipe (Pemasukan/Pengeluaran), kategori, tanggal transaksi, dan catatan opsional.
- **Manajemen Kategori:** Menyediakan kategori bawaan (misal: Makanan, Transportasi, Gaji) dan mengizinkan pengguna membuat kategori kustom yang terisolasi hanya untuk mereka sendiri.

C. Ringkasan Dashboard Finansial

Pusat visualisasi data untuk membantu pengguna mengambil keputusan finansial dengan cepat.

- **Grafik Arus Kas (Cash Flow Summary):** Visualisasi komponen total pemasukan vs total pengeluaran menggunakan komponen grafik berbasis React yang ringan dan responsif.
- **Indikator Limit Anggaran (Budget Limit Indicator):** Komponen *progress bar* visual yang menunjukkan persentase pengeluaran terhadap batas anggaran yang ditentukan pada kategori tertentu, berubah warna menjadi merah jika pengeluaran melebihi 90% limit.

3. Multi-Tenant User Flow Diagram Deskriptif

Alur di bawah ini menggambarkan perjalanan pengguna dengan penekanan pada bagaimana keamanan isolasi data ditegakkan di setiap langkah operasional:

Langkah 1

Registrasi / Login

Pengguna memasukkan kredensial melalui frontend React.js. Supabase Auth memvalidasi kredensial dan menghasilkan JWT (JSON Web Token) yang aman, berisi metadata pengguna termasuk `auth.uid()` yang unik sebagai ID Tenant.

Langkah 2

Akses Dashboard & Pengambilan Data

Aplikasi React meminta data ke Supabase API Gateway dengan melampirkan JWT. PostgreSQL RLS secara otomatis menginterseptor query dan menyaring data: HANYA baris data yang memiliki `tenant_id` cocok dengan `auth.uid()` yang akan dikembalikan ke client.

Langkah 3

Input & Pembuatan Transaksi Baru

Pengguna mengisi formulir transaksi (Nominal, Kategori, Tipe). Di latar belakang, sistem secara otomatis menyisipkan ID Tenant aktif ke dalam payload data sebelum dikirimkan.

Langkah 4

Validasi Level Database (RLS Enforcement)

Supabase mengevaluasi aturan WITH CHECK pada policy RLS. Jika `tenant_id` pada data baru sesuai dengan `auth.uid()` token pengguna, transaksi berhasil disimpan. Jika terjadi manipulasi payload di sisi client, database langsung menolak dengan error 403 Forbidden.

Langkah 5

Logout Aman

Pengguna menekan tombol keluar, aplikasi menghapus token JWT dari session client-side. Akses langsung menuju Supabase terputus total seketika.

4. Database Schema PostgreSQL for Supabase

Arsitektur database menggunakan satu database PostgreSQL bersama, di mana kolom `tenant_id` bertindak sebagai pemisah logis mutlak antar tenant.

A. Tabel: `profiles`

Menyimpan informasi dasar pengguna/tenant yang terikat langsung dengan Supabase Auth.

Nama Kolom	Tipe Data	Aturan / Constraint	Deskripsi
id	uuid	Primary Key, References <code>auth.users(id)</code>	ID unik tenant yang berasal dari sistem Supabase Auth.
full_name	text	Not Null	Nama lengkap pengguna platform.
updated_at	timestamp with time zone	Default: <code>now()</code>	Waktu pembaruan data terakhir.

B. Tabel: categories

Menyimpan kategori transaksi yang bersifat spesifik untuk masing-masing tenant.

Nama Kolom	Tipe Data	Aturan / Constraint	Deskripsi
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>	ID unik untuk setiap entri kategori.
tenant_id	uuid	Not Null, References <code>profiles(id)</code>	Penanda kepemilikan tenant demi isolasi data ketat.
name	text	Not Null	Nama kategori (misal: "Makanan", "Investasi").
budget_limit	numeric	Default: 0	Batas anggaran maksimal untuk kategori tersebut.
created_at	timestamp with time zone	Default: <code>now()</code>	Waktu pembuatan entri kategori.

C. Tabel: transactions

Menyimpan data keuangan historis milik tenant.

Nama Kolom	Tipe Data	Aturan / Constraint	Deskripsi
id	uuid	Primary Key, Default: <code>gen_random_uuid()</code>	ID unik untuk setiap transaksi keuangan.
tenant_id	uuid	Not Null, References <code>profiles(id)</code>	Penanda kepemilikan tenant (Kunci utama isolasi).
category_id	uuid	Not Null, References <code>categories(id)</code>	Relasi kunci asing ke tabel kategori terkait.
amount	<code>numeric(15,2)</code>	Not Null, Check (<code>amount >= 0</code>)	Nominal transaksi finansial (mendukung desimal).
type	text	Not Null, Check (<code>type IN ('income', 'expense')</code>)	Jenis transaksi: Pemasukan atau Pengeluaran.
transaction_date	timestamp with time zone	Not Null, Default: <code>now()</code>	Tanggal dan waktu terjadinya transaksi keuangan.
notes	text	Nullable	Catatan atau deskripsi tambahan dari tenant.

5. Security & RLS Blueprint

Untuk memastikan data antar tenant tidak bocor (*data leakage mitigation*), kita wajib mengaktifkan Row Level Security (RLS) pada tabel `categories` dan `transactions`. Supabase menyediakan fungsi bawaan `auth.uid()` yang mengekstrak ID pengguna secara aman langsung dari JWT token yang dikirimkan oleh aplikasi React.js.

```

-- 1. Mengaktifkan Row Level Security (RLS) pada tabel
ALTER TABLE categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;

-- =====
-- POLICY UNTUK TABEL: categories
-- =====

-- Kebijakan SELECT: Tenant hanya bisa melihat kategorinya sendiri
CREATE POLICY "Users can view their own categories"
ON categories FOR SELECT
USING (tenant_id = auth.uid());

-- Kebijakan INSERT: Tenant hanya bisa memasukkan data dengan tenant_id miliknya
CREATE POLICY "Users can insert their own categories"
ON categories FOR INSERT
WITH CHECK (tenant_id = auth.uid());

-- Kebijakan UPDATE: Tenant hanya bisa mengubah kategorinya sendiri
CREATE POLICY "Users can update their own categories"
ON categories FOR UPDATE
USING (tenant_id = auth.uid())
WITH CHECK (tenant_id = auth.uid());

-- Kebijakan DELETE: Tenant hanya bisa menghapus kategorinya sendiri
CREATE POLICY "Users can delete their own categories"
ON categories FOR DELETE
USING (tenant_id = auth.uid());

-- =====
-- POLICY UNTUK TABEL: transactions
-- =====

-- Kebijakan SELECT: Tenant hanya bisa melihat transaksinya sendiri
CREATE POLICY "Users can view their own transactions"
ON transactions FOR SELECT
USING (tenant_id = auth.uid());

-- Kebijakan INSERT: Tenant hanya bisa mencatat transaksi atas namanya sendiri
CREATE POLICY "Users can insert their own transactions"
ON transactions FOR INSERT
WITH CHECK (tenant_id = auth.uid());

-- Kebijakan UPDATE: Tenant hanya bisa memperbarui transaksinya sendiri
CREATE POLICY "Users can update their own transactions"

```

```
ON transactions FOR UPDATE
USING (tenant_id = auth.uid())
WITH CHECK (tenant_id = auth.uid());

-- Kebijakan DELETE: Tenant hanya bisa menghapus transaksinya sendiri
CREATE POLICY "Users can delete their own transactions"
ON transactions FOR DELETE
USING (tenant_id = auth.uid());
```

**Catatan Arsitektur: Dengan cetak biru RLS di atas, meskipun ada upaya peretasan di sisi frontend yang mencoba mengubah tenant_id menjadi milik orang lain saat melakukan request, Supabase API Gateway melalui PostgreSQL engine akan langsung menolak operasi tersebut di level database secara real-time.*

6. Constraints & Tech Stack Summary

- **Frontend Ecosystem:** Dikembangkan menggunakan **Vite + React.js** dengan arsitektur berbasis komponen UI yang modular dan efisien.
- **Backend & DB Infrastructure:** Memanfaatkan layanan serverless penuh dari **Supabase Cloud**. Operasi CRUD berjalan langsung via REST/GraphQL API Supabase yang diamankan ketat oleh kebijakan RLS di tingkat database.
- **Hosting & CI/CD Pipeline:** Seluruh kode sumber diintegrasikan ke repositori git (GitHub/GitLab) dan di-deploy langsung ke platform **Vercel** dengan pipeline otomatisasi CI/CD untuk memastikan siklus perilisan MVP berjalan cepat dan andal.